UCL Immortals na AWS

Projeto final · Desenvolvimento de Aplicações Distribuídas · Opção 1 · Parte 1 — José Lessa · outubro de 2026

1 Visão geral da aplicação

O UCL Immortals é um jogo de futebol para navegador e celular. O jogador monta um time histórico por draft, escolhendo uma carta de jogador por vez, define técnico, formação e tática, e disputa uma competição no formato da Liga dos Campeões (fase de liga e mata-mata). As partidas são simuladas a partir dos atributos das cartas, da química entre os jogadores, do técnico e da tática escolhida. Entre uma rodada e outra existem loja, mercado de transferências, trocas entre jogadores e projetos do clube que evoluem o time.

Modos de jogo

- **Solo:** uma pessoa contra times controlados pelo computador.
- **Online:** uma sala com até oito pessoas. Cada uma monta o próprio time ao mesmo tempo, todos disputam a mesma competição, se enfrentam nas rodadas e podem negociar jogadores entre si. Quem cria a sala é o anfitrião e controla o andamento (iniciar, avançar rodada, reiniciar, transferir o comando).
- **Com conta:** qualquer um dos dois modos, mas com o resultado registrado. A conta tem perfil, histórico de competições, ranking geral por pontos, recordes, 40 conquistas, eventos com recompensas, amigos, convites para salas e curtidas de perfil.

O que acontece no servidor

O servidor é a autoridade do jogo online e das contas. O navegador só envia ações ("escolhi esta carta", "jogar a rodada", "comprar este pacote"); o servidor valida cada ação, aplica as regras e devolve o novo estado para todos da sala. Resultado das partidas, créditos, pontos de ranking, recordes e conquistas são sempre calculados e gravados do lado do servidor. Isso garante que todos os jogadores de uma sala vejam a mesma competição e que ninguém consiga alterar o próprio resultado pelo navegador. É esse conjunto (estado compartilhado entre vários clientes, comunicação em tempo real e dados persistentes que precisam ficar consistentes) que torna o jogo um bom caso para a disciplina.

2 Objetivo e escopo do projeto

O jogo já existe e está em uso numa hospedagem própria. Para o projeto será feita uma **segunda implantação, completa e independente, na AWS**, a partir de um ramo separado do código. A versão que está no ar não muda; as duas rodam em paralelo, cada uma com seu banco. O trabalho do projeto é a adaptação do jogo à infraestrutura da AWS: servidor da aplicação em máquina virtual, banco de dados relacional gerenciado, armazenamento de objetos para os arquivos estáticos e funções serverless para as tarefas que não precisam acontecer durante a requisição do jogador.

3 Arquitetura

O jogador acessa um único endereço HTTPS, atendido pela instância EC2. Ela entrega a interface, responde às requisições da API e mantém a conexão em tempo real das salas online. Tudo o que precisa durar (contas, histórico, ranking) vai para o banco no RDS, que fica numa sub-rede sem acesso pela internet. As imagens do jogo ficam no S3 e são baixadas pelo navegador direto de lá. As tarefas que podem acontecer depois da resposta ao jogador (avaliar conquistas, arquivar dados antigos, limpar sessões vencidas) rodam em funções Lambda, chamadas pelo servidor ou por agendamentos do EventBridge.

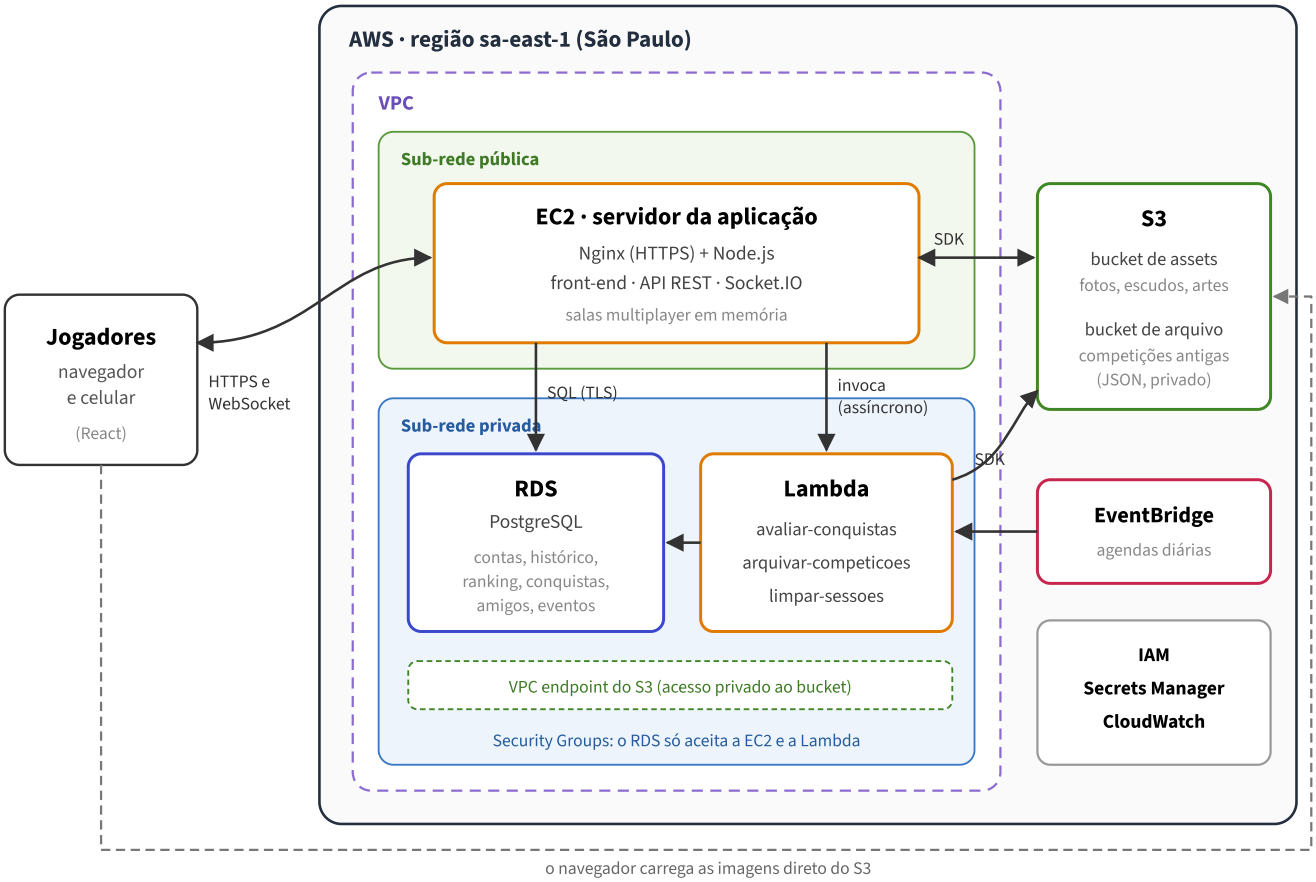


Figura 1. Arquitetura da implantação na AWS. Setas contínuas: chamadas entre componentes. Seta tracejada: imagens carregadas pelo navegador direto do S3.

4 Componentes e responsabilidades

4.1 Amazon EC2: servidor da aplicação

Uma instância Linux (Amazon Linux 2023, tamanho t3.small) com um IP fixo (Elastic IP). Nela rodam o **Nginx**, que recebe o tráfego na porta 443, cuida do certificado HTTPS e repassa para a aplicação, e o **servidor Node.js**, mantido em execução pelo systemd. O servidor tem três partes:

- **Front-end:** entrega a aplicação React já compilada (HTML, JavaScript e CSS). As imagens não passam por aqui; ficam no S3.
- **API REST das contas** (16 rotas): cadastro, login e sessão por cookie; perfil e aparência; histórico de competições; ranking por pontos e recordes; conquistas e mural do perfil; eventos e molduras; amigos, presença e convites para salas; curtidas de perfil.
- **Tempo real (Socket.IO sobre WebSocket):** criação e entrada em salas, draft simultâneo, simulação das rodadas, loja, mercado, trocas entre jogadores e controles do anfitrião, com cerca de 30 tipos de mensagem. Cada sala existe na memória do servidor enquanto a competição acontece, porque a simulação precisa responder rápido para todos. Ao terminar, o resultado de cada jogador com conta é gravado no RDS.

4.2 Amazon RDS: banco de dados (PostgreSQL)

Banco relacional gerenciado: a AWS cuida da máquina, das atualizações e dos backups, e a aplicação se conecta como a qualquer PostgreSQL. Instância db.t3.micro, numa sub-rede privada, com backup automático diário (retenção de 7 dias) e conexão criptografada. A estrutura do banco é criada por migrações versionadas, aplicadas na implantação. Os dados se dividem assim:

Área	Tabelas	O que guarda
Contas e acesso	`users`, `sessions`, `auth_rate_limits`	Usuários, senhas (hash PBKDF2), sessões ativas e limite de tentativas de login
Perfil	`profiles`, `profile_stats`, `profile_likes`	Nome, foto, capa, moldura, mural, estatísticas da carreira e curtidas
Competições	`competition_history`, `competition_records`	Cada competição concluída (fase, dificuldade, pontos, retrato do time) e recordes de jogadores
Progressão	`user_achievements`, `user_frames`	Nível de cada conquista e molduras ganhas em eventos
Social	`friendships`, `room_invitations`, `account_presence`	Amizades, convites para salas online e quem está online

4.3 Amazon S3: armazenamento de objetos

Dois buckets, ambos acessados pelo AWS SDK:

- **Bucket de assets** (leitura pública): todos os arquivos estáticos do jogo, cerca de 1.900 arquivos e 44 MB, entre fotos de jogadores, escudos de clubes, fotos de técnicos, estádios, cartas especiais, avatares e capas de perfil, além das artes de conquistas, características, táticas, loja, pacotes, projetos, funções, dificuldades e molduras. Um script de implantação usa o SDK para enviar os arquivos novos ou alterados, com o tipo de conteúdo e o tempo de cache certos. A aplicação monta os endereços das imagens a partir da URL do bucket.
- **Bucket de arquivo** (privado): o retrato completo de competições antigas, em JSON. Cada competição concluída guarda o time final e os jogos, o que ocupa bastante espaço; depois de 30 dias esse retrato sai do banco e vai para o bucket. Quando o jogador abre uma competição antiga no histórico, o servidor busca o arquivo com o SDK.

4.4 AWS Lambda e Amazon EventBridge: tarefas serverless

As funções rodam em Node.js, na mesma VPC do banco, e reaproveitam o código de regras do jogo. Elas tiram do servidor da aplicação o trabalho que não precisa acontecer enquanto o jogador espera:

Função	Disparo	O que faz
avaliar-conquistas	Invocada pela EC2 (assíncrona) ao salvar uma competição	Lê o histórico do jogador, recalcula as 40 conquistas e o progresso nos eventos ativos, grava os níveis alcançados e as molduras desbloqueadas
arquivar-competicoes	EventBridge, todo dia de madrugada	Move para o bucket de arquivo os retratos com mais de 30 dias e marca no banco que foram arquivados
limpar-sessoes	EventBridge, todo dia	Apaga sessões vencidas, registros antigos de presença e de tentativas de login

4.5 Serviços de apoio

- **VPC e Security Groups:** separam a parte pública (EC2) da privada (RDS e Lambda) e definem quem pode falar com quem (seção 6).
- **IAM:** cada componente tem um papel com só as permissões de que precisa.
- **Secrets Manager:** guarda o usuário e a senha do banco; a EC2 lê o segredo ao iniciar, e nada fica escrito no código.
- **CloudWatch:** recebe os logs do servidor Node.js e das funções Lambda, mostra métricas de uso (CPU da EC2, conexões do RDS, erros das funções) e dispara alarmes por e-mail.

5 Comunicação entre as partes

De	Para	Como	Para quê
Navegador	EC2	HTTPS (REST)	Interface, login, perfil, histórico, ranking, conquistas, amigos
Navegador	EC2	WebSocket (Socket.IO)	Salas online: ações dos jogadores e atualizações de estado em tempo real
Navegador	S3	HTTPS	Baixar imagens do jogo
EC2	RDS	PostgreSQL com TLS	Ler e gravar contas, competições e progressão
EC2	S3	AWS SDK	Ler competições arquivadas
EC2	Lambda	AWS SDK (invocação assíncrona)	Pedir a avaliação de conquistas após cada competição
EventBridge	Lambda	Agendamento	Rodar o arquivamento e a limpeza diários
Lambda	RDS / S3	PostgreSQL com TLS / SDK pelo VPC endpoint	Ler o histórico, gravar resultados e mover arquivos

6 Rede e segurança

A VPC tem uma sub-rede pública e duas privadas (em zonas de disponibilidade diferentes, exigência do RDS). Só a EC2 fica na pública. O banco e as funções ficam nas privadas, sem rota para a internet; as funções chegam ao S3 por um **VPC endpoint**, sem sair da rede da AWS.

Security Group	Entrada permitida
sg-aplicacao (EC2)	Portas 80 e 443 de qualquer lugar; porta 22 (SSH) só do IP do administrador
sg-banco (RDS)	Porta 5432 só de sg-aplicacao e sg-funcoes
sg-funcoes (Lambda)	Nenhuma (as funções só iniciam conexões)

Permissões do IAM: o papel da EC2 pode ler o segredo do banco, ler o bucket de arquivo e invocar *avaliar-conquistas*; o papel das funções pode acessar a VPC, ler e gravar no bucket de arquivo e escrever logs. Ninguém usa chaves de acesso fixas: as credenciais vêm dos papéis. No lado da aplicação, as senhas são guardadas com hash, a sessão usa cookie HttpOnly e Secure, e todas as regras do jogo são validadas no servidor.

7 Fluxos principais

Partida online

1. O anfitrião cria a sala; o servidor gera um código e guarda a sala em memória. Os outros entram pelo código ou por convite de amigo.
2. Cada ação de um jogador (escolher carta, jogar rodada, comprar, negociar) vai pelo WebSocket. O servidor valida, aplica a regra e envia a atualização para todos da sala, que veem sempre o mesmo estado.
3. No fim da competição, o servidor grava o resultado de cada jogador com conta no RDS e invoca *avaliar-conquistas* para cada um.

Fim de uma competição solo

1. O navegador envia o relatório final; o servidor recalcula a fase alcançada e os pontos (não confia no valor enviado) e grava a competição e os recordes no RDS.
2. O servidor invoca *avaliar-conquistas* sem esperar a resposta e devolve o resultado ao jogador na hora.
3. A função grava as conquistas e molduras desbloqueadas; elas aparecem quando o jogador abre o perfil.

Consulta ao histórico

1. O jogador abre uma competição no histórico. Se ela tem menos de 30 dias, o retrato vem do RDS.
2. Se já foi arquivada, o servidor busca o JSON no bucket de arquivo e devolve o mesmo conteúdo, sem diferença para quem vê.

8 Implantação e operação

- **Código:** ramo próprio no repositório, com a camada de banco adaptada para PostgreSQL e um servidor Node.js de produção. A versão no ar continua no ramo principal, sem alteração.
- **Implantação:** um script compila o front-end, envia os assets novos para o S3 pelo SDK, copia o servidor para a EC2, aplica as migrações pendentes no RDS e reinicia o serviço. As funções Lambda são empacotadas e publicadas separadamente.
- **Backups:** backup automático diário do RDS e versionamento ativado no bucket de arquivo.
- **Monitoramento:** logs e métricas no CloudWatch, com alarmes de CPU alta na EC2, falta de espaço no RDS e erros nas funções.

9 Requisitos da disciplina

Requisito	Como é atendido
Instância com a aplicação	EC2 com Nginx e o servidor Node.js (front-end, API e tempo real)
SGBD separado	RDS com PostgreSQL, alternativa aceita no enunciado à segunda instância
Blob via SDK	S3 com os assets do jogo e o arquivo de competições, ambos acessados pelo AWS SDK
Serviço serverless	Três funções Lambda, uma chamada pela aplicação e duas agendadas pelo EventBridge
Recursos adicionais	VPC com sub-redes e Security Groups, VPC endpoint, IAM, Secrets Manager e CloudWatch

10 Limitações e próximos passos

Como as salas online vivem na memória do servidor, esta proposta usa uma única instância de aplicação: se ela reiniciar, as salas em andamento se perdem (os dados das contas, que ficam no RDS, não). Para crescer, o caminho seria colocar um Application Load Balancer na frente de várias instâncias com Auto Scaling e guardar o estado das salas num serviço compartilhado, como o ElastiCache (Redis), que também serviria de canal entre as instâncias do Socket.IO. Uma distribuição pelo CloudFront na frente do bucket de assets reduziria o tempo de carregamento das imagens. Esses pontos ficam como evolução, fora do escopo mínimo do projeto.